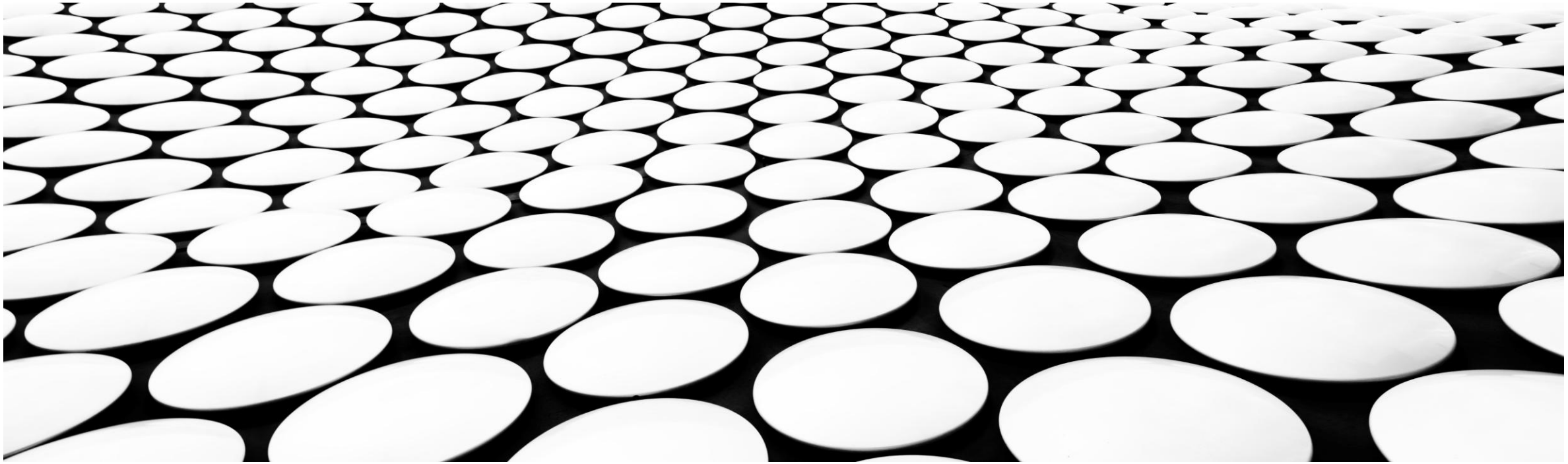
LIQUIBASE



INTRODUCTION

- **Liquibase** is an open-source database schema change management tool.
- It allows developers to manage and version control database changes in a consistent and repeatable way.
- Supports multiple databases (e.g., MySQL, PostgreSQL, Oracle, SQL Server, etc.).

INSTALLATION ON RHEL

Liquibase Opensource Installation

```
sudo rpm --import https://repo.liquibase.com/liquibase.asc
sudo yum install -y yum-utils
sudo yum-config-manager --add-repo https://repo.liquibase.com/repo-liquibase-com.repo
sudo yum install Liquibase
which liquibase
liquibase --version
```

INSTALLATION ON WINDOWS

Liquibase Opensource Installation

Download Liquibase Windows installaer from Downloads page

https://www.liquibase.com/download

Follow the prompts of Installation Wizard.

Java is also installed as Liquibase needs Java

KEY CONCEPTS

Changelog

- A changelog is a file that contains a list of changesets.
- It acts as the master file for tracking all database changes.

ChangeSet

- A changeset is a single unit of change in the database.
- Defined in a file (XML, JSON, YAML, or SQL) and contains one or more changes.
- Each changeset has:
- A unique id.
- An author (to identify who made the change).
- A filepath (to track the source file).

KEY CONCEPTS

Rollbacks

- Liquibase supports rollback operations to undo changes.
- Rollbacks can be:
- Automatic: Liquibase generates rollback SQL for certain operations (e.g., createTable → dropTable).
- Manual: Custom rollback SQL can be defined.

Snapshots

- creating a record of the database's current structure
- Captures the current state of a database.
- useful for tracking changes
- comparing database states

KEY CONCEPTS

Schema Differences

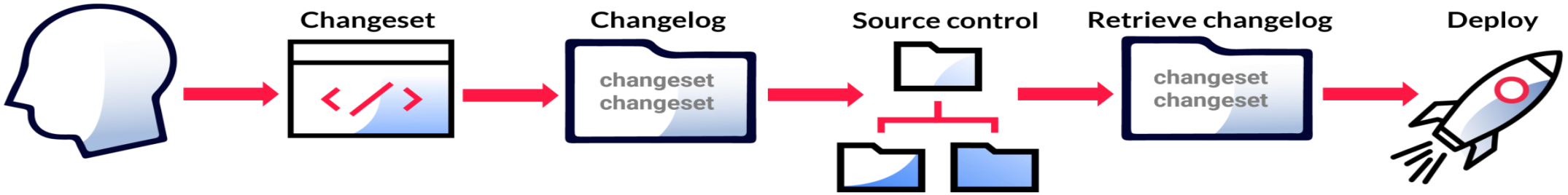
- Compares two databases
- Compares current state of database against previously created snapshot
- Allows to identify
  - Missing objects
  - Unexpected objects
- Generates Changelog

CHANGELOG

- Is a text file to list all the changes to database
- SQL, XML, JSON and YAML formats are supported
- Can be saved in any source control tool
- Consist of meta information and change sets

CHANGE SETS

- Is a basic unit of change
- All changesets are saved in Changelog file
- Each changeset should have “author” and “id”
- Combination “author” and “id” identifies the changeset uniquely within Changelog
- Changelog File, author and id identifies changes made to database uniquely



CHANGELOG & CHANGE SETS XML

```
<?xml version="1.0" encoding="UTF-8"?>
<databaseChangeLog
        xmlns="http://www.liquibase.org/xml/ns/dbchangelog"
        xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
        xmlns:pro="http://www.liquibase.org/xml/ns/pro"
        xsi:schemaLocation="http://www.liquibase.org/xml/ns/dbchangelog
http://www.liquibase.org/xml/ns/dbchangelog/dbchangelog-latest.xsd
    http://www.liquibase.org/xml/ns/pro http://www.liquibase.org/xml/ns/pro/liquibase-pro-latest.xsd ">
<changeSet author="abdul.sami" id="8">
  <createTable tableName="pet_table">
      <column name="id" type="int">
        <constraints primaryKey="true"/>
      </column>
      <column name="breed" type="varchar(50)">
        <constraints nullable="false"/>
      </column>
  </createTable>
</changeSet>
</databaseChangeLog>
```

CHANGELOG & CHANGE SETS SQL

```
--liquibase formatted sql
--changeset abdul.sami:8
create table pet_table (
   id int primary key,
   breed varchar(50) not null
);
```

LIQUIBASE PROPERTIES FILE

- Properties file contain connection information to the database
- Database credentials
- Change Log file name
- Additionally, it can contain reference database details

LIQUIBASE UPDATE COMMANDS

- Applies the changes from ChangeLog to Database
- Uses connection details from file liquibase.properties if present
- Connection properties can be given manually using option --url and --ChangeLogFile
- Can also be executed to generate SQL only to examine before actually Applying

```
liquibase --url=jdbc:yourDatabaseUrl --changeLogFile=path/to/changelog.xml update

liquibase   update

Liquibase-sql  --url=jdbc:yourDatabaseUrl --changeLogFile=path/to/changelog.xml update

Liquibase-sql    update
```

LIQUIBASE STATUS COMMANDS

- Reports the changes applied and pending changes from a ChangeLog file
- Useful validate if any changes are pending

```
liquibase --url=jdbc:yourDatabaseUrl --changeLogFile=path/to/changelog.xml status

liquibase   status
```

LIQUIBASE HISTORY COMMANDS

- Reports all the changes which are applied
- Gets the data from DatabaseChangeLog table

```
liquibase --url=jdbc:yourDatabaseUrl --changeLogFile=path/to/changelog.xml history

liquibase   history
```

LIQUIBASE SNAPSHOT COMMANDS

- Creates the snapshot current database structure
- Can be used as reference to validate any changes in the schema

```
liquibase --url=jdbc:mysql://localhost:3306/mydb --output-file=snapshot.yaml snapshot --snapshot-format=yaml
```

LIQUIBASE ROLLBACK COMMANDS

- Allows to rollback to previous state of database
- Should be used cautiously as it can have undesirable effects
- Can be rollbacked to certain number of changes (RollbackCount 3)
- Can be rollbacked to certain date (RollbackToDate yyy-mm-dd)
- Can be rollbacked to previous created tag (liquibase tag <tag>)
- Tag are saved in DatabaseChangelog table

```
liquibase   RollbackCount 3

liquibase   RollbackToDate '2025-03-05'

liquibase   Rollback <tag>
```

LIQUIBASE DIFF COMMANDS

- Finds the difference between current database schema and a snapshot or other database schema
- Reports Missing objects
- Reports unexpected objects
- Can be used to create changelog to sync the target database schema with reference database for snaphot

```
liquibase --url=jdbc:yourDatabaseUrl --referenceUrl=jdbc:referenceDatabaseUrl diff

liquibase --url=jdbc:yourDatabaseUrl --referenceUrl=snapshot.xml diff


liquibase --url=jdbc:yourDatabaseUrl --referenceUrl=jdbc:referenceDatabaseUrl --
changeLogFile=path/to/changelog.xml diffChangeLog
```